

ORCA Simulation Implementation Report

1. Problem Statement

The goal is to implement a solution for **reciprocal n-body collision avoidance**, where a group of independent agents must select velocities that minimize collision risk while prioritizing movement towards their respective goals. The agents operate without communication, relying only on local observations.

2. Implementation Architecture

The simulation is built in Python using an Object-Oriented programming approach with two primary classes:

- **Robot Class:** Represents the individual agent, storing state (e.g., position, velocity, radius) and calculating the desired goal movement (`compute_preferred_velocity`).
- **ORCA Simulator Class:** Manages the overall environment, the discrete time step ($dt=0.1$), and orchestrates the collision-avoidance calculations for all agents.

3. Collision Avoidance Approach (ORCA Core Logic)

The simulation resolves the problem by implementing a simplified version of the Optimal Reciprocal Collision Avoidance (ORCA) technique, which involves a continuous **sensing and acting cycle**:

3.1. Generating Constraints

The simulator utilizes two functions to generate the required geometric constraint for each agent pair:

- **`compute_velocity_obstacle(robot_a, robot_b)`:** This function serves to detect the proximity of an imminent collision threat within the time horizon (τ). It calculates an **avoidance vector**, which represents the necessary velocity adjustment to prevent collision.
- **`compute_orca_halfplane(robot_a, robot_b)`:** This function generates the final constraint by calculating the normal and the boundary point of the ORCA half-plane. It enforces the **reciprocal principle** by setting the boundary point based on: $\text{point} = v_a + 0.5 * u$, ensuring each robot takes half the responsibility for the avoidance maneuver.

3.2. Velocity Selection (Linear Programming)

The `linear_program` function determines the final velocity, by solving an optimization problem against the set of collected ORCA half-plane constraints:

1. **Objective:** Find the velocity that minimizes the distance to the preferred velocity:
2. **Constraint Check:** The function first attempts to use the preferred velocity. If it violates any half-plane constraints, it falls back to a **sampling approach**.
3. **Sampling and Selection:** It samples various velocities within the maximum speed circle (`max_speed`) and selects the valid sampled velocity that has the minimum Euclidean distance. This serves as a practical, albeit non-optimal, solution to the problem originally solved by a specialized Quadratic Program.

4. Simulation Execution

The `ORCASimulator.update()` method controls the flow:

1. **Parallel Computation:** It iterates through all robots, calculating the `new_velocity` for each based on the constraints of all other agents.
2. **Synchronous Update:** After all new velocities are determined, the position and velocity states of all robots are updated simultaneously using the time step (`dt`).
3. **Visualization:** The simulation generates **snapshots** and a final **trajectory comparison** plot, allowing for visual analysis of how speed and direction vectors change dynamically to achieve collision-free navigation.